

Recorrido del código

Qué hace cada parte de los archivos del motor

Playa de Estacionamiento — Grupo 13

Archivos: **euler.py** y **simulacion.py**

Este documento recorre el **código real** de los dos archivos del motor, explicando qué hace cada función y cada bloque, en lenguaje claro. Los nombres en **este color** son nombres reales que aparecen en el código.

Parte 1 — euler.py

Es un archivo cortito con una sola tarea: calcular cuánto tarda un auto en pagar. Tiene dos datos fijos arriba y una función principal.

Los datos fijos del archivo

```
UMBRAL = {"Grande": 180, "Pequeño": 130, "Utilitario": 130}
MAX_PASOS = 1_000_000
```

UMBRAL es una tabla que dice hasta qué número tiene que llegar el cálculo según el tipo de auto: 180 para los Grandes y 130 para el resto. **MAX_PASOS** es un tope de seguridad (un millón) para cortar el cálculo si por algún parámetro raro nunca llegara a destino, y así evitar que el programa se quede pensando para siempre.

La función `integrar(C, tipo, T, h)`

Es el corazón del archivo. Recibe cuatro datos y devuelve el tiempo de cobro. Primero **revisa que los datos sean válidos** y, si no lo son, corta con un mensaje claro:

```
if tipo not in UMBRAL:
    raise ValueError("Tipo de auto desconocido...")
if not h > 0:
    raise ValueError("El paso h debe ser > 0...")
if C < 0:
    raise ValueError("C no puede ser negativo...")
```

Después hace el cálculo. Arranca dos contadores en cero: **D** (lo que se va acumulando) y **t** (el tiempo). Y repite un paso una y otra vez **mientras D no haya llegado al umbral**:

```
D = 0.0; t = 0.0; tabla = []
while D < umbral:
    dD = C + 0.2 * T + t * t      # cuánto crece en este paso
    tabla.append((t, D, dD))      # se anota la fila
    D = D + h * dD                # se suma al acumulado
    t = t + h                     # avanza el tiempo
```

En cada vuelta: calcula **dD** (cuánto crece ahora), **anota la fila** en **tabla** para poder mostrarla después, le suma ese crecimiento a **D** y adelanta **t** un paso **h**. Un contador **pasos** vigila que no se pase de **MAX_PASOS**.

Cuando **D** alcanza el umbral, el bucle termina, se agrega la fila final (el momento del corte) y la función devuelve tres cosas: **t** (el tiempo de cobro en minutos), el **umbral** usado y la **tabla** completa con todos los pasos.

En resumen, **integrar** "llena un balde de a cucharadas" hasta el borde y te dice cuántas cucharadas (tiempo) hicieron falta, guardando el registro de cada una.

Parte 2 — simulacion.py

Es el archivo grande: el motor de la simulación. Lo recorreremos por bloques, en el mismo orden en que están escritos.

Bloque 1 — Parámetros y estado

Arriba de todo están los **parámetros** (los valores con los que se juega): `N_LUGARES` (cuántos lugares), `MEDIA_LLEGADA` (cada cuánto llega un auto), `PRECIO`, `PROB_TIPO` y `PROB_HORAS` (los porcentajes), y los datos de Euler `T_EULER` y `H_EULER`.

Más abajo están las **variables de estado**: la "memoria viva" de la simulación. Las principales:

Variable	Qué guarda
<code>lugares</code>	La lista de los N lugares, cada uno con su estado (Libre/Ocupado) y qué auto tiene.
<code>autos</code>	Todos los autos presentes en el sistema en este momento.
<code>zona_cobro / cola_cobro</code>	La caja (un auto por vez) y la fila de los que esperan pagar.
<code>eventos</code>	La agenda de cosas por pasar, ordenada por hora (el "heap").
<code>reloj</code>	La hora simulada actual.
<code>recaudacion, acum_ocupacion</code>	Acumuladores de plata y de ocupación para las estadísticas.
<code>contador_llegaron, contador_abandonos</code>	Cuántos autos llegaron y cuántos se fueron por estar lleno.

Bloque 2 — `validar_parametros()`

Antes de simular, esta función revisa que **todo cierre** y, si encuentra problemas, los junta y corta con un mensaje. Por ejemplo: que los lugares sean al menos 1, que la media de llegadas sea mayor que 0, que los porcentajes sumen 100% y que los precios no sean negativos.

Es una red de seguridad: hace que el motor **falle rápido y explicando el motivo**, en lugar de colgarse o devolver números sin sentido si alguien carga datos imposibles.

Bloque 3 — Las variables aleatorias (los sorteos)

Cuatro funciones traducen números al azar en decisiones:

- `_take(...)`: entrega el próximo número al azar. Puede usar una lista fija preparada (para comparar con Excel) o números realmente aleatorios.
- `gen_llegada()`: convierte el azar en "minutos hasta el próximo auto" con una fórmula exponencial. Incluye un blindaje para que el número quede siempre entre 0 y 1 y la fórmula no falle.
- `gen_tipo()`: recorre los porcentajes de tipo y devuelve Pequeño, Grande o Utilitario (la "ruleta" de tipos).
- `gen_horas()`: igual, pero para decidir si el auto se queda 1, 2, 3 o 4 horas.

Bloque 4 — La agenda de eventos

`push_evento(...)` anota un evento futuro en la agenda; `siguiente_evento()` saca el más próximo; y `tiempo_de(tipo)` busca cuándo es el próximo evento de cierto tipo (sirve para mostrar "próxima llegada" en la tabla). Por debajo usa un `heap`, que mantiene todo ordenado por hora automáticamente.

Bloque 5 — Ayudantes y la foto del estado

`lugar_libre()` busca el primer lugar disponible y `ocupados()` cuenta cuántos hay ocupados. Después, `guardar_fila(...)` es clave: cada vez que pasa algo, **saca una "foto" completa del sistema** (qué auto llegó, el estado de cada lugar, la cola, la plata acumulada, etc.) y la agrega a `vector_estado`. Esa lista de fotos es exactamente la tabla que se ve en pantalla.

Bloque 6 — Los procesadores de eventos

Acá está la lógica principal: una función por cada tipo de evento.

Función	Qué hace cuando ocurre el evento
<code>procesar_llegada_auto()</code>	Programa la próxima llegada; si hay lugar libre, ubica al auto, le sortea tipo y horas y calcula su importe; si está lleno, lo cuenta como abandono. Guarda la fila.
<code>_iniciar_cobro(aid, C)</code>	Mete un auto en la caja: llama a <code>euler.integrar</code> para saber cuánto tardará en pagar y programa su evento de fin de cobro.
<code>procesar_fin_estacionamiento(...)</code>	Libera el lugar del auto. Si la caja está libre, lo manda a pagar; si está ocupada, lo pone en la cola.
<code>procesar_fin_cobro(aid)</code>	El auto paga (suma a la recaudación) y se va. Si había alguien esperando, entra el primero de la cola.

Bloque 7 — El bucle principal `simular(...)`

Es el director de orquesta. Primero llama a `validar_parametros()`, programa la primera llegada, y después entra en un bucle que **repite hasta que se acabe el tiempo**:

```
while iteracion < MAX_ITER and reloj < tiempo_max:
    tiempo, _, tipo, eid = siguiente_evento() # el más próximo
    reloj = tiempo                          # adelanta el reloj
    if tipo == "llegada_auto":               procesar_llegada_auto()
    elif tipo == "fin_estacionamiento":      procesar_fin_estacionamiento(eid)
    elif tipo == "fin_cobro":                 procesar_fin_cobro(eid)
```

En cada vuelta saca el evento más próximo, mueve el reloj a esa hora y llama a la función que corresponde. El `MAX_ITER` es otro tope de seguridad para que nunca quede en bucle infinito.

Bloque 8 — Resultados y reinicio

`estadisticas(...)` calcula al final los números que importan: recaudación, porcentaje de utilización y porcentaje de abandonos. Y `resetear_estado()` deja todas las variables como al principio, para poder volver a simular desde cero sin reiniciar el programa.

Para llevarse: `euler.py` calcula un solo número (el tiempo de cobro). `simulacion.py` maneja todo lo demás: los parámetros, su validación, los sorteos, la agenda de eventos, la lógica de cada evento, el bucle del reloj y las estadísticas finales.